Buenas prácticas de la programación en C++ moderno

Elaborado originalmente por Christopher Torres

Taller de Programación 2026-1

- *"Código limpio, reutilizable, y escalable"*
- Ser coherente y consistente con tu propio estilo, escribe lo mismo siempre de la misma manera para evitar mezclar estilos.
- Orden de importaciones y funcionamiento de este:
    - primero #ifndef [Nombre del .h]
    - segundo #define [Nombre del .h]
    - includes con formato <>
    - includes con formato ".h"
    - #defines de macros (#define UP 1) (ocasionalmente utilizados)
    - typedefs (no utilizados)
    - using namespace std (para aumentar legibilidad)
    - ### Código ###
    - último #endif

*La sentencia inicial se lee: if not define "archivo", define "archivo"… fin de la definición. Esto se hace debido a que en algún punto de las importaciones se redefinen cabeceras y esto evita ese conflicto interno del programa. No es obligatorio usarlo pero es muy útil y recomendado.

Una cabecera debería verse de la siguiente manera (faltaría escribir #endif en la última línea del archivo):

```
#ifndef Graph_h
#define Graph_h

#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
#include "Edge.h"
#include "Vertex.h"

using namespace std;
```

```
#endif /* Graph_h */
```

Otra cosa importante de las importaciones, es que hay que evitar redundancia (también produce conflictos). A modo de ejemplo considerando la clase “Graph”, esta poseerá archivos como “Graph.h” y “Graph.cpp”, entendiendo que el .h es la declaración de la clase con sus métodos y el .cpp la implementación de esta. Ahora, dentro del archivo “Graph.cpp” necesitamos importar la clase “Node.h” para poder crear un grafo, por lo que este “Node.h” deberá ir incluído en el archivo “Graph.h” y NO dentro del archivo “Graph.cpp”, ya que en este último, sólo se debe restringir a incluir “Graph.h”. En resumen, las importaciones deben ir entre las propias cabeceras y no dentro de los .cpp. Existen casos donde se puede producir un acoplamiento de importaciones de manera que una clase incluye a otra y viceversa (esto se traduce en un error de compilación, por lo que hay que tener un poco de orden para estas cosas). Esto mismo se aplica a las librerías que se utilicen en el programa. Si una clase ya trae previamente importada <iostream>, es redundante volver a incluirla en el archivo donde se está importando.

- Estructuración de clases:
    - Es de suma importancia que se comente la definición de la clase a modo de abstracción de esta. En simples palabras, se debe comentar

qué es lo que representa la clase dentro del lenguaje natural (aplicar POO de manera correcta).

Un ejemplo de esto es (por encima de la clase):

```
/*
    * Graph class:
    * This class is used to create a random graph with the given
    * number of vertices and the given probability of edges.
    * The graph is represented by a vector of vectors and the nodes
    * are represented by the index of the main vector.
*/
```

Dentro de las clases se deben distinguir (principalmente) dos tipos de acceso a datos: public y private.

- private se usa (generalmente) para atributos.
- public se usa (generalmente) para métodos.

A modo de ejemplo:

```
class Graph {
    private:
        int numberOfNodes;
        vector<vector<int>> graphContainer;
    public:
        // Constructor
        Graph(int nodes);
        void createGraph(double probability);

        // Destructor
        void cleanContainer();

        // Getters
        int getNumberOfNodes();
        vector<int> getAdjacentNodes(int node);

        // Setters
        void addEdge(int node, int otherNode);

        // Other methods
        bool doesEdgeExist(int firstNode, int secondNode);
        void saveGraph(string fileName);
        void print();
};
```

- Comentarios dentro de los .h
    - Estos deben ser idealmente de una línea utilizando // (comentario simple)
      En el ejemplo anterior, no se ve ningún comentario porque todos los nombres de atributos y métodos son sumamente declarativos, por lo que no hace falta redundar en comentarios. Este sería un código muy bien hecho, aunque esto sucede porque no tiene métodos tan complejos, que, eventualmente necesitarán de una mejor explicación.
    - Cabe mencionar que, los comentarios deben ser una forma de estructurar piezas de código, así como los espaciados o saltos de línea con tal de que esto ayude a la legibilidad del código en sí (como se dividen en getters, setters, other methods, etc.).

- Comentarios dentro de los .cpp
    - Dado que los .cpp sólo contienen las implementaciones de los métodos, estos son tratados como “funciones”, por lo que deberán ser documentados. Hay muchos estilos de cómo comentar, pero todos

siguen una misma lógica: una descripción que menciona qué hace el método, sus argumentos y su retorno. A modo de ejemplo:

```
#include "Graph.h"

/*
    * Method: Constructor->Graph
    * Description: This method is used to create a graph with
    * the given number of vertices.
    * Parameters:
    *  - nodes: The number of vertices in the graph.
    * Return:
    *  - void.
*/
Graph::Graph(int nodes){
    numberOfNodes = nodes;
    for (int i = 0; i < numberOfNodes; i++)
        graphContainer.emplace_back();
}
```

- Además, cuando las funciones son más complejas, de vez en cuando es necesario comentar dentro de los métodos siempre y cuando éstos no entorpezcan la legibilidad y sean un aporte al entendimiento.

"Un programa sin comentarios puede ser difícil de entender (excepto cuando el código es muy claro), y por otro lado un programa con demasiados comentarios no deja ver el código y es también muy difícil de entender. El objetivo de los comentarios es facilitar la comprensión del código a otra persona, por lo que es más importante la calidad de los comentarios que la cantidad".

- Tamaño de las funciones
  - Idealmente estas no deben sobrepasar el tamaño de una ventana con una tipografía normal (básicamente que se pueda leer toda la función y no tener que hacer scroll). Esto se logra encapsulando submétodos o dividiendo funcionalidades al programa.
- Tests Unitarios
  - Estos no son necesariamente códigos complejos, por lo que solamente se componen de los includes necesarios y una función main() para ejecutar sus funciones.

- Se pueden comentar los resultados esperados o hacer una estructuración propia (varia mucho dependiendo del contexto).

- Estilos de escritura
  - K&R:

```
if (K&R){
    std::cout << “K&R” << std::endl
}
```

  - ANSI C++

```
if (ANSI C++)
{
    std::cout << “ANSI C++” << std::endl
}
```

  - GNU

```
if (GNU)
    {
        std::cout << “GNU” << std::endl
    }
```

Actualmente, el estilo predominante en C++ es K&R debido a que los demás suelen ocupar líneas innecesarias que, en una función compleja estorban la lectura.

REFERENCIAS:

- https://docplayer.es/62899663-Guia-de-estilo-y-buenas-practicas-de-programacion-en-c-c.html
- http://informatica.uv.es/iiguia/AED/laboratorio/Estilocpp.pdf
- https://r0d.developpez.com/articles/buenas-practicas/#LI